

10주차 개인요약본

🕒 과제 종류	기타
📎 자료	Euron 10주차 발표자료.pdf

5. 토큰화

- **토큰화**: 자연어 처리 모델을 만들기 위해 말뭉치(대규모 자연어)를 개별 단어나 문장 부호와 같은 토큰으로 나누는 과정이 필요하다.
 - 토큰나이저(텍스트 문자열을 토큰으로 나누는 알고리즘/소프트웨어)를 사용
- **문장의 토큰화 예시**
 - 입력: '형태소 분석기를 이용해 간단하게 토큰화할 수 있다.'
 - 결과: ['형태소', '분석기', '를', '이용', '하', '어', '간단', '하', '게', '토큰', '화', '하', 'ㄹ', '수', '있', '다', '.']
- 토큰나이저 구축 방법: 공백 분할, 정규표현식 적용, 어휘 사전 적용, 머신러닝 활용 등

5.1 단어 및 글자 토큰화

- **단어 토큰화**: 텍스트 데이터를 의미 있는 단위인 단어로 분리하는 작업으로, 가장 일반적인 토큰화 방법.
 - OOV와 접사 처리 한계가 있음.
- **글자 토큰화**: 문장을 글자 단위로 나누는 방식.
 - 비교적 작은 단어 사전을 구축할 수 있어 컴퓨터 자원을 효율적으로 사용할 수 있음.
- **자모(jamo) 라이브러리 활용**
 - 접사와 문장 부호의 의미 학습이 가능
 - OOV(Out-of-Vocabulary)를 획기적으로 줄일 수 있으나, 개별 토큰의 의미가 없어 자연어 모델이 의미를 조합해야 하는 단점이 있음. (다의어나 동음이의어가 많은 도메인에서 구별이 어려워짐)

5.2 형태소 토큰화

- **형태소 토큰화**: 텍스트를 형태소 단위(실제로 의미를 가진 최소 단위)로 나누는 방법으로, 언어의 문법과 구조를 고려해 단어를 분리하고 이를 의미 있는 단어로 분류한다.
 - 자립 형태소, 의존 형태소
- **형태소 토큰화 라이브러리**
 - **KoNLPy**: 한국어 자연어 처리를 위해 개발된 라이브러리
 - 명사 추출, 형태소 분석, 품사 태깅 등의 기능 제공
 - Okt, 꼬꼬마(Kkma), 코모란(Komoran) 등 다양한 형태소 분석기 지원
 - **NLTK**: 주로 영어 자연어 처리를 위해 개발(다른 언어도 가능)
 - **spaCy**: 머신 러닝 기반의 자연어 처리 라이브러리로, 빠른 속도와 높은 정확도를 목표
 - NLTK는 학습 목적으로 자연어 처리에 대한 다양한 알고리즘 예제 제공
 - spaCy는 효율적인 처리 속도와 높은 정확도를 제공하는 것을 목표로 함

5.3 하위 단어 토큰화

기존 형태소 분석기의 문제점

- OOV(Out-of-Vocabulary): 신규어/신조어/도메인 특화 용어 처리 취약
- 형태소 경계 의존: 띄어쓰기/품사 태깅 오류 전파
- 언어/도메인 편향: 특정 코퍼스 · 사전 설계에 과적합
- 유연성 부족: 다국어/도메인 확장 시 재설계/재학습 비용 큼

- 기존 형태소 분석기의 문제점을 해결하기 위해 등장함
- 너무 미세(문자)도, 너무 거칠(단어)지도 않게 중간 단위로 분해하여, OOV 감소, 어휘 축소(메모리 및 연산 절감), 형태 변형(어미 및 접사) 대응
- 자주 함께 나타나는 문자열 묶기 → 서브워드 단위로 모델링
 - ex) 토크나이저 → "토크", "##나이", "##저"

▪ 원문: 시보리도 짱짱하고 허리도 어병하지만구 죠하효

▪ 결과: ['시', '보리', '도', '짱짱하고', '허리', '도', '어', '병하지만구', '죠', '하', '효']

▪ 원문: 진짜 멋있네요! 돈줄날만 하네요.

▪ 결과: ['진짜', '멋있네요', '!', '돈줄날', '만', '하네요', '.']

- 주요 기법

- **BPE**: 문자에서 시작해 가장 자주 출현하는 인접 쌍을 병합하는 과정을 반복 (빈도 기반)
 - 산출물: 병합 규칙 + 어휘
- **SentencePiece**: 구글에서 개발한 오픈소스 하위 단어 토큰라이저 라이브러리
 - 언어 비의존적(공백 비의존)으로, raw 텍스트를 바로 학습할 수 있다. (BPE/Unigram 알고리즘을 지원함)
 - **Korpora(코포라)** : 한국어 오픈소스 말뭉치들의 다운로드와 전처리를 라이브러리로 불러와서 쉽게 사용할 수 있게 해주는 라이브러리
- **WordPiece**: BERT 계열에서 사용함. 언어 모델 우도/점수를 사용해서 어휘를 선택한다.
- **Hugging Face tokenizers**: 러스트 기반 고성능, 파이썬 바인딩을 제공한다. 대용량 코퍼스에서 빠른 학습이 가능하고, transformers와 저장 포맷/로더가 호환되기 때문에 사용된다.

토큰라이저 학습 공통 프로세스

1. 데이터 수집: 도메인 대표성/크기/클린
2. 정규화: lowercasing, NFD/NFKC, 특수문자 규칙
3. 프리토큰화: 공백/ByteLevel/Metaspace 등
4. 모델 학습: BPE/WordPiece/Unigram, vocab 크기 결정
5. 포스트프로세싱: [CLS]/[SEP] 삽입 등 템플릿 처리
6. 평가: 토큰 길이, 커버리지, OOV, Perplexity 대리 지표
7. 저장/배포: 버전 고정, 재현성/호환성 체크

6. 임베딩

- **텍스트 벡터화**: 텍스트 → 숫자 벡터로 변환
 - 원핫 인코딩, 빈도 벡터화, TF-IDF 등
 - 단점: 벡터의 희소성이 큼, 의미 반영 어려움
- **워드 임베딩**: 단어를 고정된 길이의 실수 벡터로 표현 → 의미 관계 반영
 - Word2Vec, fastText 등
 - 단점: 문맥 반영 어려움 → 동적 임베딩 등장

6.1 언어 모델(Language Model)

- 입력된 문장으로 각 문장을 생성할 수 있는 확률을 계산하는 모델 → 다음 단어를 예측함
 - 완성된 문장 단위로 확률을 예측하는 것은 어렵다
- **자기회귀 모델(Autoregressive)**: 이전 단어(조건부확률)로 다음 단어 예측
- **통계적 언어모델(Statistical LM)**: 언어의 통계적 구조를 이용해 문장이나 단어의 시퀀스를 생성하거나 분석함. 마르코프 체인 기반, 데이터 희소성 문제 발생.
 - **마르코프 체인**: 빈도 기반의 조건부 확률 모델로, 이전 상태와 현재 상태 간의 전이 확률을 이용해 다음 상태를 예측

6.2 N-gram

- N개의 연속된 단어를 하나의 단위로 취급 → 특정 단어 시퀀스가 등장할 확률 추정
- 관용구나 패턴 분석에 유용함

6.3 TF-IDF

- 단어의 중요도를 수치화하는 대표 통계적 방법
- BoW에 가중치를 부여함
 - BoW(Bag-of-Words): 문서/문장에 들어가는 단어의 빈도수 기록 (중복 허용)
- **TF-IDF의 한계점**
 - 문맥과 단어의 순서를 고려하지 않음 → 문장 생성이나 문장 순서가 중요한 작업에 부적합
 - 단어의 의미를 반영하지 않음 → TF-IDF 벡터는 중요도만 나타내며, 단어 간 의미적 유사성을 표현하지 못함